

Quantum coin toss in Cirq

```
import cirq
qubit = cirq.LineQubit(0)
circuit = cirq.Circuit(
    cirq.H(qubit),
    cirq.measure(qubit, key='result')
)
simulator = cirq.Simulator()
result = simulator.run(circuit, repetitions=100)
histogram = result.histogram(key='result')
print("Heads:", histogram.get(0, 0), "Tails:", histogram.get(1, 0))
```

The sample output from running the above code using the Cirq quantum simulator may look like this:

Heads: 47 Tails: 53

 **Note:** The exact numbers will vary each time you run the simulation, as quantum randomness ensures that the counts of heads (0) and tails (1) are close but not necessarily equal for 100 repetitions. Both examples show how a quantum computer can simulate a fair coin toss, with results fluctuating due to quantum randomness.

The evolution of quantum software tools

The trajectory of quantum software has moved at breakneck speed, evolving from low-level hardware control to high-level abstraction in less than a decade. Understanding this evolution is crucial for appreciating how accessible the field has become.

Phase 1: The assembly era (QASM): In the early stages of quantum computing, programming was akin to writing Assembly language. Researchers wrote OpenQASM (quantum assembly language), specifying instructions at the physical pulse level. A programmer had to know exactly which physical qubit on the chip was connected to another to perform a CNOT gate. This required deep knowledge of microwave physics and hardware topology, creating a massive barrier to entry.

Phase 2: The SDK and circuit era: The release of IBM Qiskit (2017), Google Cirq, and Microsoft Q# marked the democratisation of quantum coding. These software development kits (SDKs) introduced the concept of the ‘quantum circuit’ as a software object. Developers could now write Python code to arrange gates abstractly. Crucially, these tools introduced the transpiler (or compiler). The transpiler acts as a translator, taking an abstract circuit and mapping it to the physical constraints of a specific device, automatically handling the routing and optimisation of gates. This allowed computer scientists to focus on algorithms (like

Grover’s or Shor’s) without worrying about the underlying qubit connectivity.

Phase 3: Hybrid orchestration and kernels: We are currently transitioning into an era of hybrid quantum-classical computing. It is now understood that quantum computers will not replace classical ones but will act as accelerators (similar to GPUs). Modern tools like Qiskit Runtime and AWS Braket Hybrid Jobs allow for the seamless interleaving of classical and quantum instructions. For example, in Variational Quantum Eigencovers (VQE)—an algorithm used in chemistry—a classical CPU optimises parameters while the QPU calculates energy states. The software stack now manages this loop automatically, reducing latency and enabling ‘serverless quantum computing’, where developers submit a job without provisioning the underlying infrastructure.

Phase 4: The future – domain-specific abstraction: The next frontier, already visible with tools like Classiq and Horizon quantum computing, is moving beyond gates entirely. In this emerging paradigm, a developer might define a high-level problem—such as “optimise this financial portfolio” or “simulate this molecule”—and the software stack will automatically synthesise the optimal quantum circuit to solve it. This shift mirrors the evolution from C++ to Python/SQL, where the ‘how’ is abstracted away, leaving the user to focus entirely on the ‘what’.

Quantum programming is an exciting frontier, blending physics, mathematics, and computer science. Whether you’re a student, a seasoned coder, or simply curious, now is the perfect time to learn the language of qubits and shape the future of computing. **END** 

References

- <https://www.bluequbit.io/blog/quantum-programming-languages>
- <https://quantumai.google/cirq/start/basics>
- <https://www.spinquantum.com/news-detail/the-ultimate-guide-to-quantum-algorithms>
- <https://quantumai.google/>
- <https://learn.microsoft.com/en-us/azure/quantum/qsharp-overview>
- <https://qiskit.github.io/qiskit-aer/>
- <https://qosf.org/>

By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd. This article expresses his views and not of the organisation he works in.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning, and cyber security.